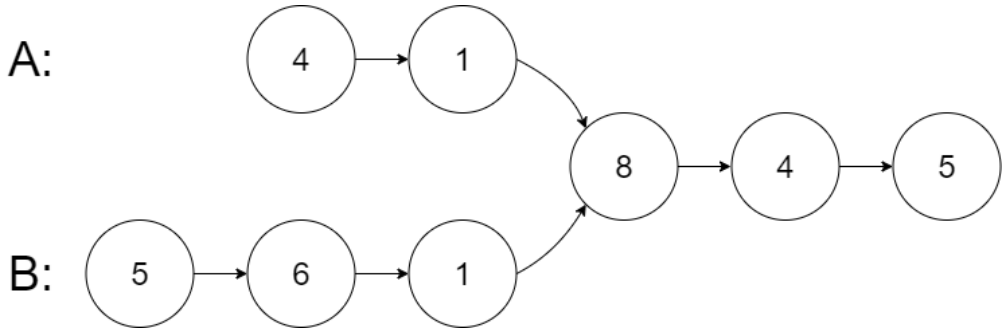
LinkList

leetcode

160.相交链表

- 给定两个单链表的头节点 `headA` 和 `headB`，找到两个单链表相交的起始节点。



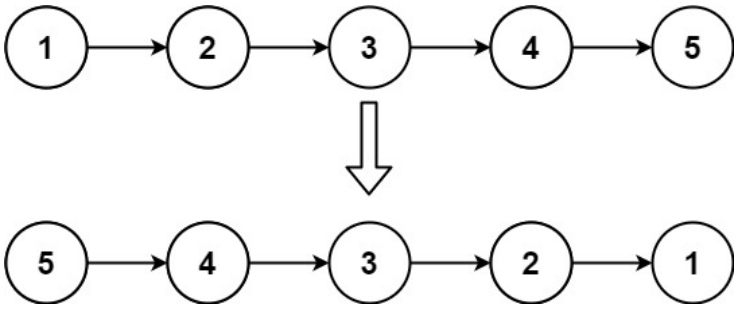
- 思路：先遍历一遍A链表，把所有的节点都加入哈希集合。再遍历一遍B链表，检查第一个在哈希集合出现的B链表结点，就是起始交点。

```
class Solution {
public:
    ListNode *getIntersectionNode(ListNode *headA, ListNode *headB) {
        unordered_set<ListNode *> visited;// 存储结点索引
        // 先遍历一遍A链表，把所有的节点都加入哈希集合
        ListNode *temp = headA;
        while( temp!=nullptr ){
            visited.insert(temp);
            temp=temp->next;
        }
        // 再遍历一遍B链表，检查第一个在哈希集合出现的B链表结点，就是起始交点
        temp = headB;
        while( temp!=nullptr ){
            if(visited.count(temp)){
                return temp;
            }
            temp=temp->next;
        }
        return nullptr;
    }
};
```

-

206.反转链表

- 给一个单链表，反转链表。

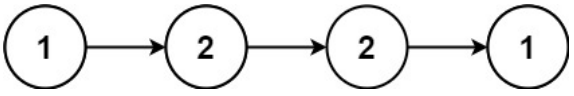


- 思路：遍历一遍链表。把每个结点的pnext指针换成前驱就好。

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
 * };
 */
class Solution {
public:
    ListNode* reverseList(ListNode* head) {
        ListNode *pre = nullptr,*post = head; // 修改post指针的后驱为pre
        while(post!=nullptr){
            ListNode *temp = post->next; // 暂存当前处理结点post的后驱
            post->next=pre;
            // pre和post均后移
            pre=post;
            post=temp;
        }
        return pre;
    }
};
```

0234.回文链表

- 判断一个链表是否为回文链表（从前向后和从后向前读都相同的序列）。



- 思路：遍历链表，把值复制到数组中，然后定义首尾双指针，遍历判断元素是否相等。

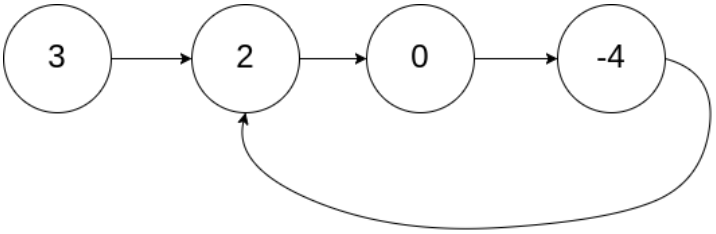
```
class Solution {
public:
    bool isPalindrome(ListNode* head) {
        vector<int> nums;
        // 将链表元素移动到数组
        ListNode * p = head;
        while( p!=nullptr ){
            nums.push_back(p->val);
            p = p->next;
        }
        // 首尾双指针，判断是否为回文序列
        for(int front = 0,end = nums.size()-1;front<end;front++,end--){
```

```
            if(nums[front]!=nums[end]) return false;
        }
        return true;
    }
};
```

-

141.环形链表

- 判断链表中是否有环。

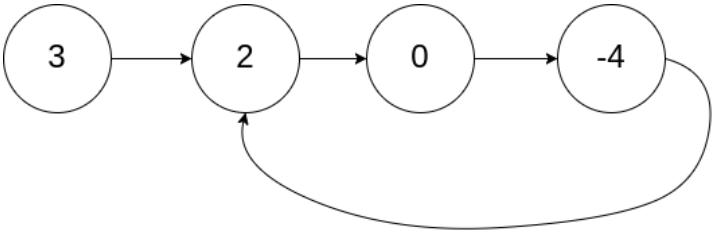


- 思路：哈希表，遍历链表，每次需要判断当前顶点是否访问过。如果访问过，则说明是环形链表。

```
class Solution {
public:
    bool hasCycle(ListNode *head) {
        unordered_set<ListNode*> visited;
        ListNode *p = head;
        while( p!=nullptr ){ // 如果是环形链表则一直无法退出该循环
            if( visited.count(p)){
                return true;
            }
            visited.insert(p);
            p=p->next;
        }
        return false;
    }
};
```

142.环形链表2

- 给定一个链表，返回链表开始入环的第一个节点。无环则返回null。
- 思路：哈希表。遍历链表并检测是否在哈希表中出现。出现则返回结点，反之返回null。




```
class Solution {
public:
    ListNode *detectCycle(ListNode *head) {
        unordered_set<ListNode*> visited;
```

```
        ListNode *p = head;
        while(p!=NULL){
            if(visited.count(p)){
                return p;
            }
            visited.insert(p);
            p=p->next;
        }
        return NULL;
    }
};
```

-

21.合并两个有序链表

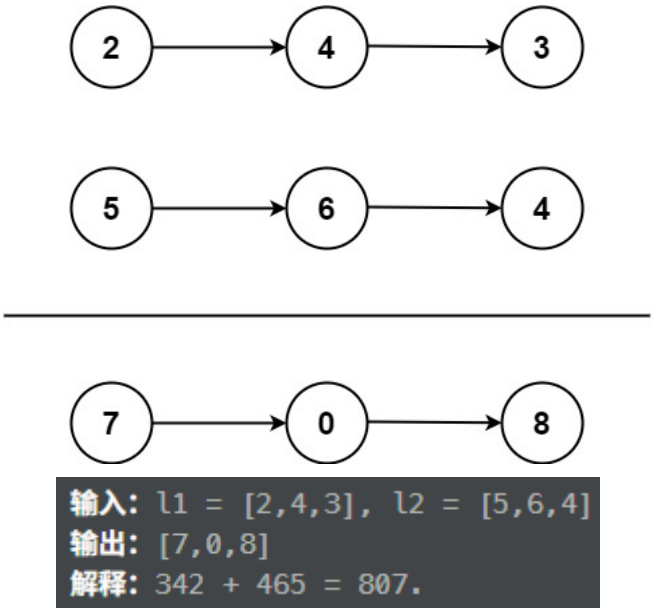
- 将两个升序链表合并为一个新的 **升序** 链表并返回。
- 思路：新建一个链表。双指针法，遍历两个链表，每次将最小的元素插入到新链表中。

```
class Solution {
public:
    // 合并两个有序链表
    ListNode* mergeTwoLists( ListNode* list1, ListNode* list2 ){
        ListNode* dummy = new ListNode(0,head);
        ListNode* cur = dummy;
        while( list1!=nullptr&&list2!=nullptr ){
            if( list1->val < list2->val ){
                cur->next = list1;
                list1 = list1->next;
            }else{
                cur->next = list2;
                list2 = list2->next;
            }
        }
        // 将剩余的链表补到后面
        cur->next = (list1?list1:list2);

        return dummy->next;
    }
};
```

-

2.两数相加

- 两个链表，存储两个数字。链表的每个结点存储一位数，而且数字是逆序排列的。




```
输入: l1 = [2,4,3], l2 = [5,6,4]
输出: [7,0,8]
解释: 342 + 465 = 807.
```

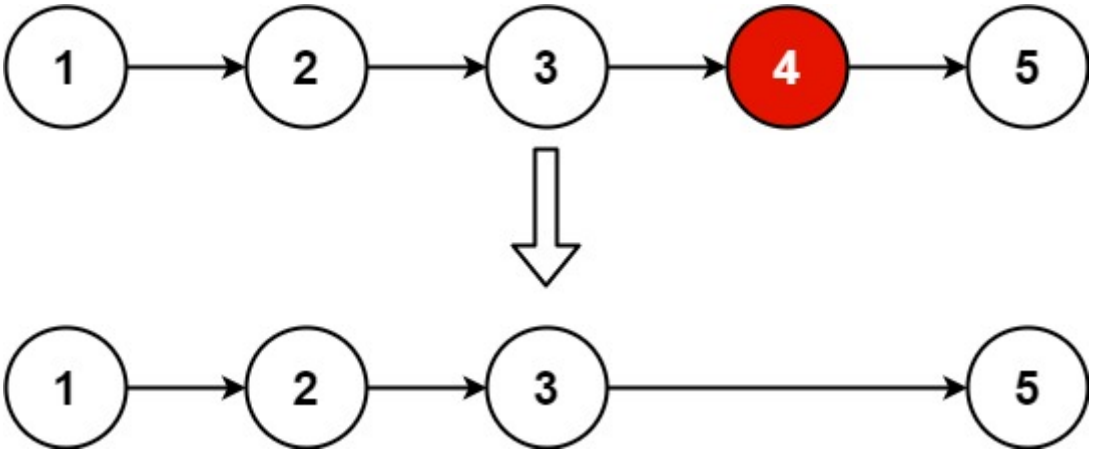
- 思路：新建一个链表来存储结果。遍历两个链表，开始求和（注意考虑进位）。

```cpp
class Solution {
public:
    ListNode* addTwoNumbers(ListNode* l1, ListNode* l2) {
        ListNode* res = new ListNode(-1);
        ListNode* p = res; // 始终指向尾部
        int carry = 0; //进位
        while(l1!=NULL||l2!=NULL){
            int sum = 0;// 每一位的相加结果
            if(l1!=NULL){
                sum += l1->val;
                l1=l1->next;
            }
            if(l2!=NULL){
                sum += l2->val;
                l2=l2->next;
            }
            if(carry) sum++;// 上一轮的进位
            carry = (sum>9)?1:0; // 这一轮的进位
            // 存储结果
            ListNode *temp = new ListNode(sum%10);
            p->next = temp;
            p=p->next;
        }
        // 处理最高位的进位
        if(carry) p->next = new ListNode(1);

        return res->next;
    }
};
```

-

19.删除链表的倒数第N个结点

- 删除链表的倒数第 `n` 个结点，并且返回链表的头结点。



- 思路：双指针法，右指针先走n步（左右指针插值为n），然后左右指针同时走，最终右指针到达左边界时，左指针指向倒数第n个结点。

```
class Solution {
public:
    ListNode* removeNthFromEnd(ListNode* head, int n) {
        ListNode *dummy = new ListNode(0,head); // 哑巴结点，处理删除第一个元素
        ListNode *l=dummy,*r=dummy;
        // 右指针先走n步
        int i = 0;
        while( i<n ){
            r=r->next;
            i++;
        }
        // 左右指针同时移动，右指针到边界，左指针到倒数第n个结点
        while( r->next!=NULL ){
            r=r->next;
            l=l->next;
        }
        // 删除结点
        ListNode *temp = new ListNode();
        temp = l->next;
        l->next = temp->next;
        delete temp;


        return dummy->next;
    }
};
```

-

24.两两交换链表中的结点

- 给定一个链表，两两交换相邻结点，然后返回头节点（不能修改链表的值）。
- 思路：假设 `swapPairs(输入)` 一定能返回一个**已经交换好**的链表头节点。只关注当前两个结点。注意终止条件。

afaaa4515fa04f1368a6c711e809a3a0

✅ 递归思维（爽）：

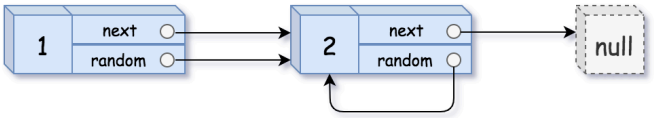
1. 现在的任务是处理 `1->2->3->4...` 。
2. 我把 `3->4...` 扔给 `swapPairs` 函数。
3. 假设它已经帮我变成了 `4->3...` 并返回了头节点 `4` 。
4. 那我现在的任务就很简单了：把 `1` 和 `2` 交换，然后把 `1` 连到那个返回来的 `4` 上。
5. 完工！返回 `2` 。
    - 我有两个节点： `head` (1) 和 `newHead` (2)。
    - 后面的事别人做好了，返回了 `rest_head` (比如 4)。
    - 我要做的动作：
        1. 让 `1` 指向 `rest_head` (`head->next = rest_head`)。
        2. 让 `2` 指向 `1` (`newHead->next = head`)。
    - 这就完成了局部的交换。

```
class Solution {
public:
    ListNode* swapPairs(ListNode* head) {
        // 终止条件，只有一个结点
        if(head==nullptr || head->next==nullptr){
            return head;
        }

        ListNode *pre = head->next;
        head->next = swapPairs(pre->next);
        pre->next = head;

        return pre;
    }
};
```

-

138.随机链表的复制

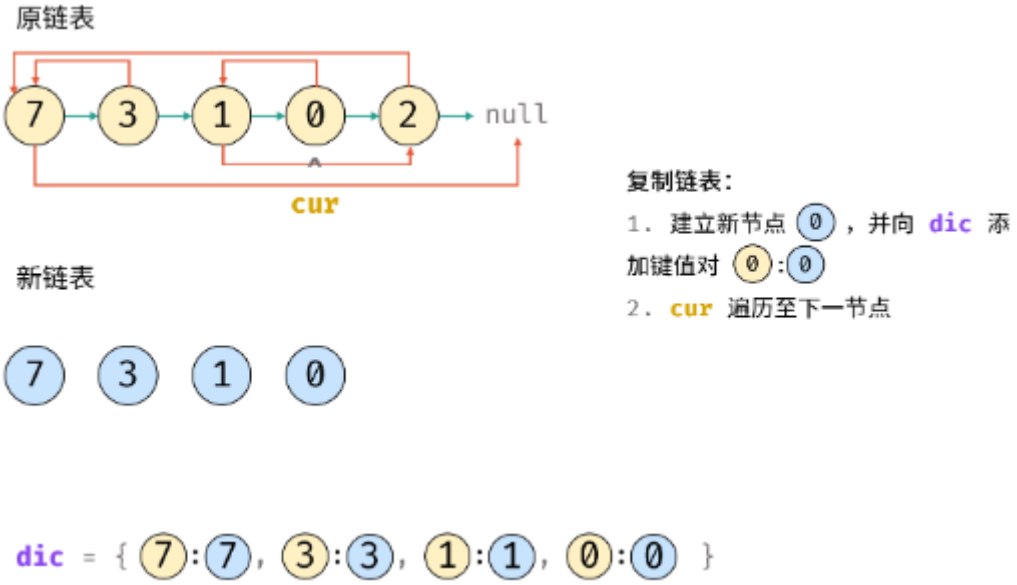
- 一个链表，每个节点包含一个额外增加的随机指针 `random`。构造这个链表的 [深拷贝](#)。 深拷贝应该正好由 `n` 个 **全新** 节点组成，其中每个新节点的值都设为其对应的原节点的值。

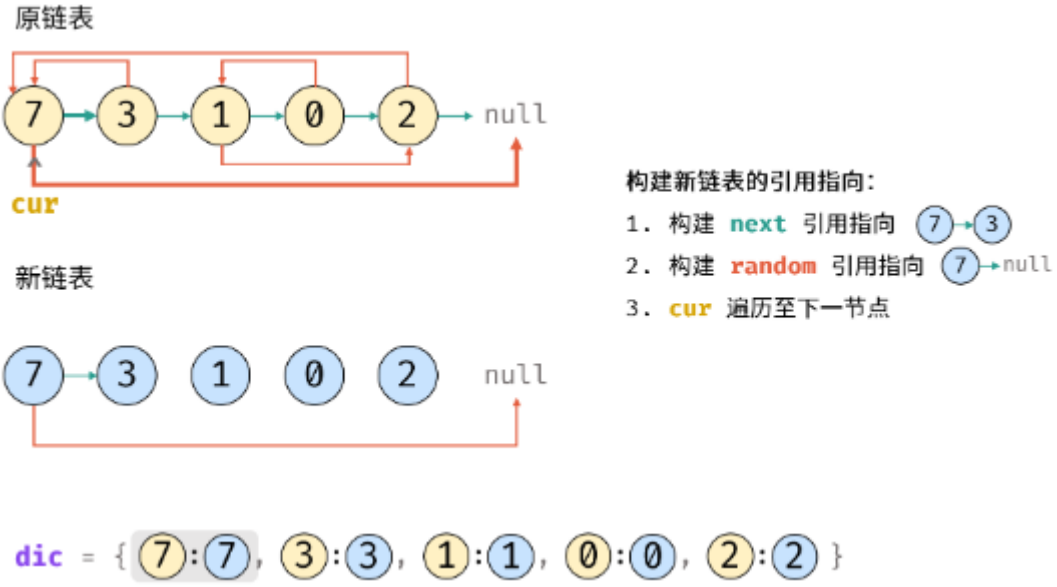


- 思路：哈希表，建立新链表结点与旧链表结点的映射。两趟遍历，一躺遍历建立值，一趟遍历建立后驱和random后驱。

代码的核心思想是 **“先建人，后认亲”** ，分为两个阶段：

1. 第一阶段（建人）：遍历旧链表，为每个旧节点创建一个对应的新节点（只复制 `val` ），并将 “旧节点 -> 新节点” 的对应关系存入哈希表（ `unordered_map` ）。此时新节点的 `next` 和 `random` 都是空的。
2. 第二阶段（认亲）：再次遍历旧链表，利用哈希表查找对应的关系，将新节点的 `next` 和 `random` 指针指向正确的新节点。





```
class Solution {
public:
    Node* copyRandomList(Node* head) {
        Node* cur = head;
        unordered_map<Node*,Node*> oldToNew;
        // 遍历链表，构建新链表结点与旧链表结点的映射（先复制值）。
        while( cur!=nullptr ){
            oldToNew[cur] = new Node(cur->val);
            cur=cur->next;
        }
        // 再次遍历链表，构建映射（复制后驱和random后驱）
        cur = head;
        while( cur!=nullptr ){
            oldToNew[cur]->next = oldToNew[cur->next];
            oldToNew[cur]->random = oldToNew[cur->random];
            cur=cur->next;
        }
        return oldToNew[head];
    }
};
```

-

876.链表的中间结点

- 找到链表的中间节点，并将链表从中间断开成两个独立的链表。
- 思路：速度差。**慢指针**（`slow`）每次走 **1** 步。**快指针**（`fast`）每次走 **2** 步。

  逻辑推导：
  因为快指针的速度是慢指针的 **2倍**，所以在相同的时间内（循环次数），快指针走过的路程也是慢指针的 **2倍**。

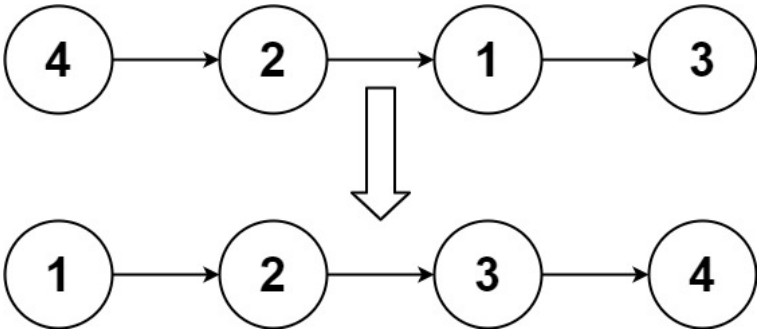
  - 当快指针跑完整个链表（到达终点 `nullptr`）时，慢指针刚好跑了一半的路程。
  - 因此，慢指针停留的位置就是链表的中点。

```
    // 找中点并断开链表
    ListNode* middleNode(ListNode* head){
        ListNode* slow = head; // 指向前半段的最后一个结点
        ListNode* fast = head->next;
        while( fast!=nullptr&&fast->next!=nullptr ){
            slow = slow->next;
            fast = fast->next->next;
        }
        ListNode* head2 = slow->next; // 第二个链表的起始点
        slow->next = nullptr; // 断开连接
        return head2;
    }
```

-

148.排序链表

- 给定链表，然后按照升序排列，返回排序后的链表。



- 思路：链表排序。归并排序，寻找中间结点，并根据中间结点将大链表分为两个小链表，分别排序，然后再合并两个有序链表。

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
 * };
 */
class Solution {
    // 找中点并断开链表
    ListNode* middleNode(ListNode* head){
        ListNode* slow = head; // 指向前半段的最后一个结点
        ListNode* fast = head->next;
        while( fast!=nullptr&&fast->next!=nullptr ){
            slow = slow->next;
            fast = fast->next->next;
        }
        ListNode* head2 = slow->next; // 第二个链表的起始点
        slow->next = nullptr; // 断开连接
        return head2;
    }

    // 合并两个有序链表
    ListNode* mergeTwoLists( ListNode* list1, ListNode* list2 ){
        ListNode* dummy = new ListNode();
        ListNode* cur = dummy;
        while( list1!=nullptr&&list2!=nullptr ){
            if( list1->val < list2->val ){
                cur->next = list1;
                list1 = list1->next;
            }else{
                cur->next = list2;
                list2 = list2->next;
            }
            cur=cur->next;
        }
        // 将剩余的链表补到后面
```

```
        cur->next = (list1?list1:list2);

        return dummy->next;
    }

public:
    ListNode* sortList(ListNode* head) {
        if ( head==nullptr || head->next==nullptr ){
            return head;
        }
        // 寻找链表的中间结点
        ListNode *head2 = middleNode(head);
        // 分治
        head = sortList(head);
        head2 = sortList(head2);
        // 合并
        return mergeTwoLists(head,head2);
    }
};
```

-

146.LRU缓存

- `LRUCache` 类:
  - `LRUCache(int capacity)` 以 **正整数** 作为容量 `capacity` 初始化 LRU 缓存
  - `int get(int key)` 如果关键字 `key` 存在于缓存中，则返回关键字的值，否则返回 `-1` 。
  - `void put(int key, int value)` 如果关键字 `key` 已经存在，则变更其数据值 `value` ；如果不存在，则向缓存中插入该组 `key-value` 。如果插入操作导致关键字数量超过 `capacity` ，则应该 **逐出** 最久未使用的关键字。

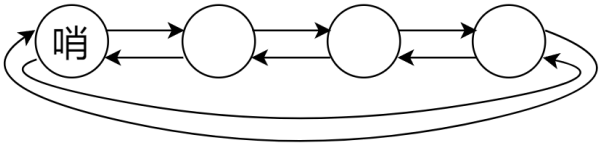
想象你有一摞书。

get：把一本书 (key) 抽出来，放在最上面。

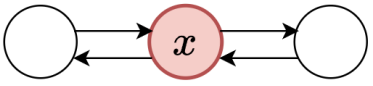
put：放入一本新书。
如果已经有这本书 (key)，
就把它抽出来放在最上面，并替换它的 $value$。
（例如把一本书的第二版替换成第三版）

如果没有这本书 (key)，就放在最上面。
如果超过 $capacity$ 本书，就把最下面的书移除。

题目要求 get 和 put 都是 $O(1)$ 时间复杂度，
这可以用**双向链表**实现，每个节点都表示一本书（key 和 $value$）。
每个节点都有两个指针 $prev$ 和 $next$ 分别指向前一个和下一个节点。
此外，链表中还包含一个**哨兵节点** $dummy$，
这可以让每个节点的 $prev$ 和 $next$ 都不为空，从而简化代码逻辑。



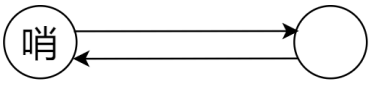
删除一个节点（抽出一本书）



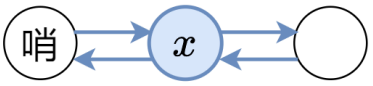
删除节点 x 之前

$$x.prev.next = x.next$$
$$x.next.prev = x.prev$$

在链表头添加一个节点（把一本书放在最上面）



插入节点 x 之前



$$x.prev = dummy$$
$$x.next = dummy.next$$
$$x.prev.next = x$$
$$x.next.prev = x$$

如何快速找到要抽出来的书？

把 key 映射到链表中的对应节点。
这可以用**哈希表**实现

这可以用**哈希表**实现。

- 思路：双向链表实现LRU缓存。

```

```

-